

Retrieval Augmented Generation

BIOC 6225

30 March, 2026

Jonathon Keeney, Ph.D.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; ^{*}New York University;
plewis@fb.com

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory have so far been only investigated for extractive downstream tasks. We

<https://arxiv.org/pdf/2005.11401>

Technical Problem: Provenance

- How did you arrive at the conclusion that you arrived at?

Technical Problem: Plasticity

- Cannot easily expand or revise memory
 - Model training is expensive and time consuming

Knowledge Problem

- Custom data is often needed to be useful
 - If a model was trained on Reddit and Wikipedia, that may not be useful enough if your domain is life sciences

Knowledge Problem

- LLM doesn't know anything about your data
 - The boundaries of an LLM's awareness is the data that it's been pretrained on
 - LLMs are static

Retrieval Augmented Generation

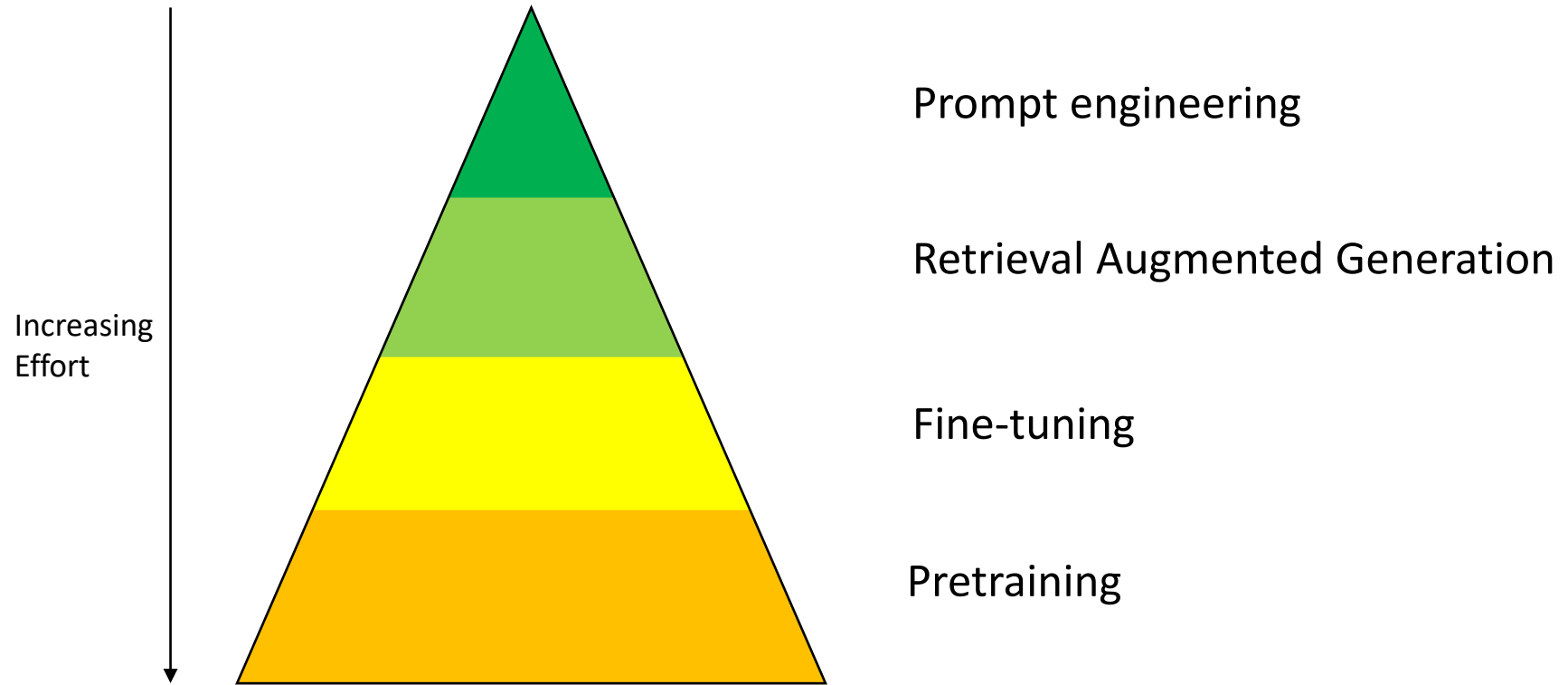
- Search engine + LLM
 - Hybrid of parametric memory + nonparametric memory

Benefits

- Factual accuracy through source citation
- Dynamic knowledge updates without retraining
- Reduced hallucinations compared to pure generation
- Domain specialization through curated knowledgebases
- Explainability via source attribution

Benefits

- Factual accuracy through source citation
- Dynamic knowledge updates without retraining
- Reduced hallucinations compared to pure generation
- **Domain specialization through curated knowledgebases**
- Explainability via source attribution



Dimension	Fine-tuning	RAG
Knowledge storage	Model weights	External DB
Update cost	High	Low
Interpretability	Lower	Higher
Latency	Lower	Higher
Application	Behavior	Knowledge

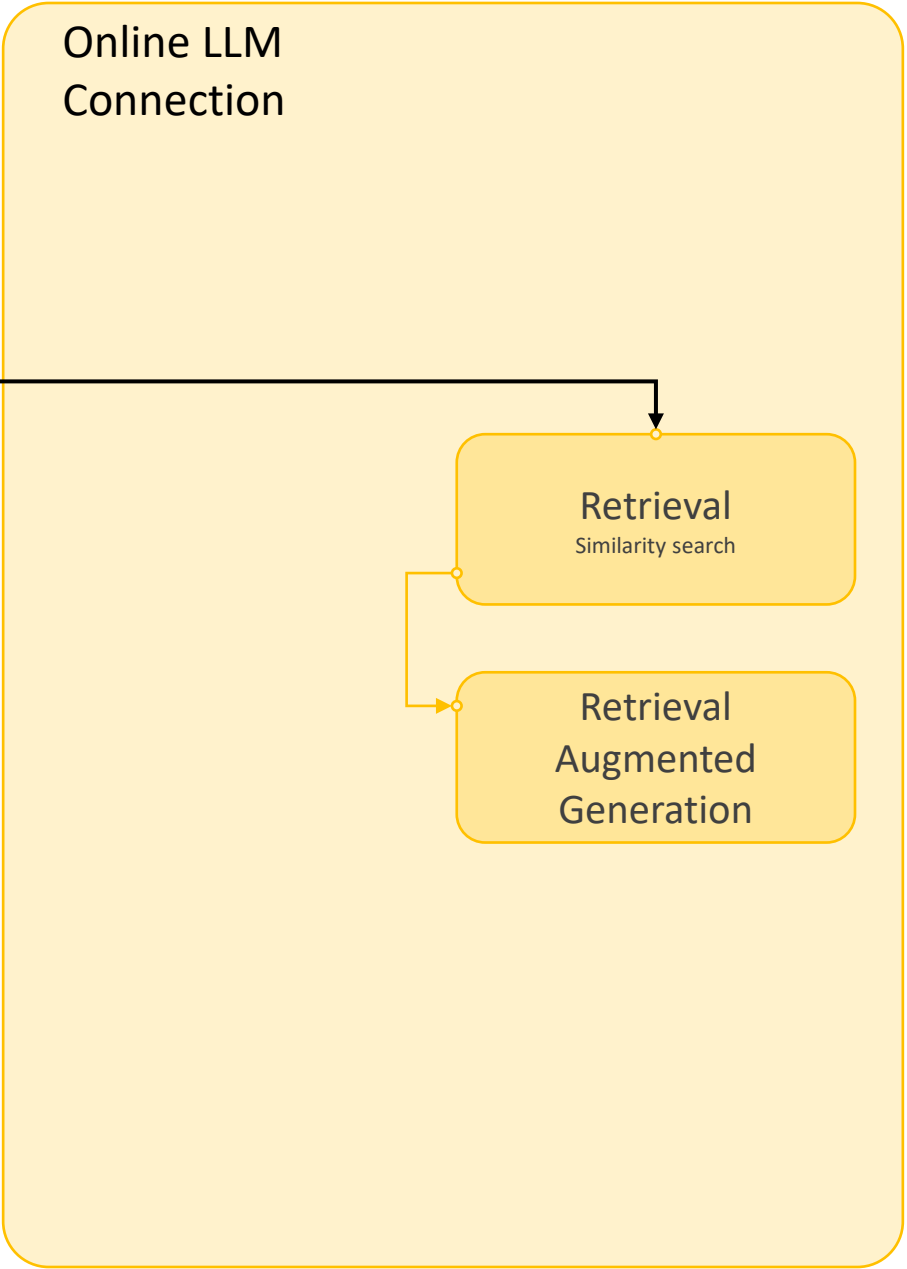
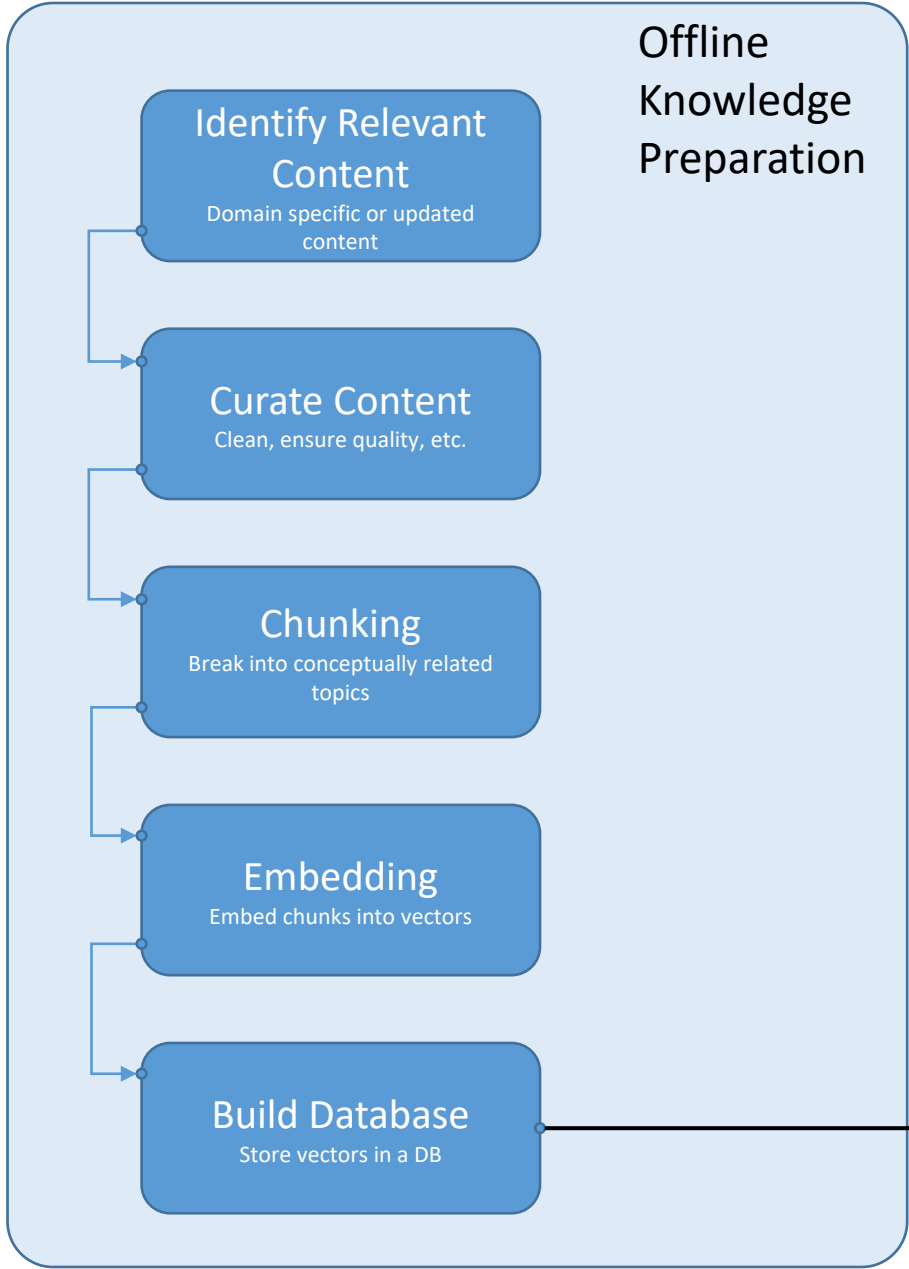
Dimension	Fine-tuning	RAG
Knowledge storage	Model weights	External DB
Update cost	High	Low
Interpretability	Lower	Higher
Latency	Lower	Higher
Best for	Behavior	Knowledge

Up-to-date papers ✓

Traceable citations ✓

Large corpus ✓

✓ Style/format control



Chunking

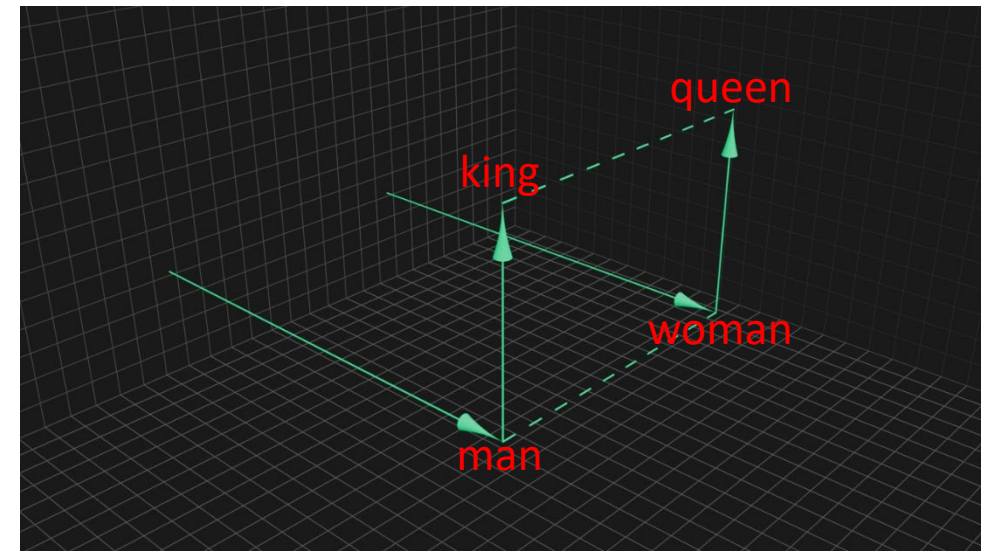
- Chunking is data segmentation
- Purpose: group by conceptually related content so that retrieval focuses only on relevant content, instead of the entire corpus of texts
- Poor chunking leads to:
 - Mixed topics
 - Lost context
 - Irrelevant retrieval

Chunking

- Previously: simple chunking by size
- Now: find conceptual boundaries
 - 2-5x improvement in retrieval
- Chunking strategy depends on content:
 - Academic papers: sections
- Ideal chunking strategy groups ideas perfectly
 - In reality: brittle

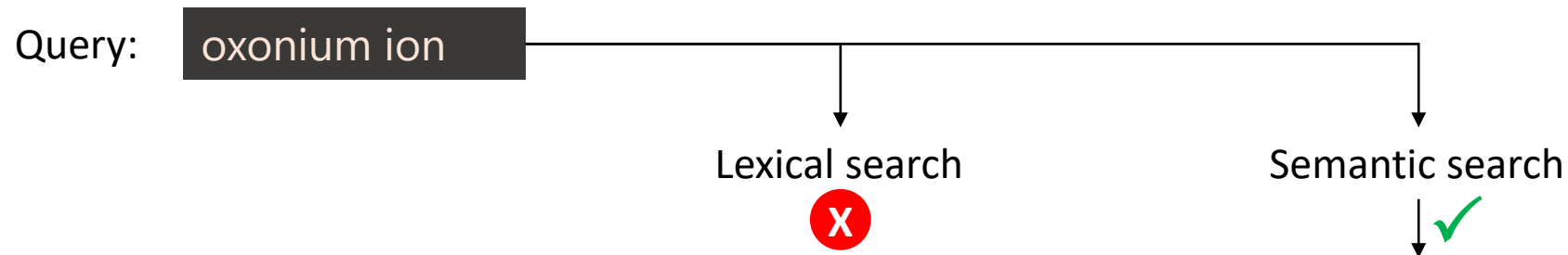
RAG Embedding

- RAG embedding $\neq$ token embedding
- Token embedding:
 - Learned representations that map tokens into vectors
 - E.g. “man” is to “king” as “woman” is to ?



RAG Embedding

- RAG embedding $\neq$ token embedding
- RAG embedding:
 - Used only for semantic search, not reasoning
 - E.g. "The mitochondria is the powerhouse of the cell" becomes [0.0231, -0.1123, 0.4471, ...]
- Example:



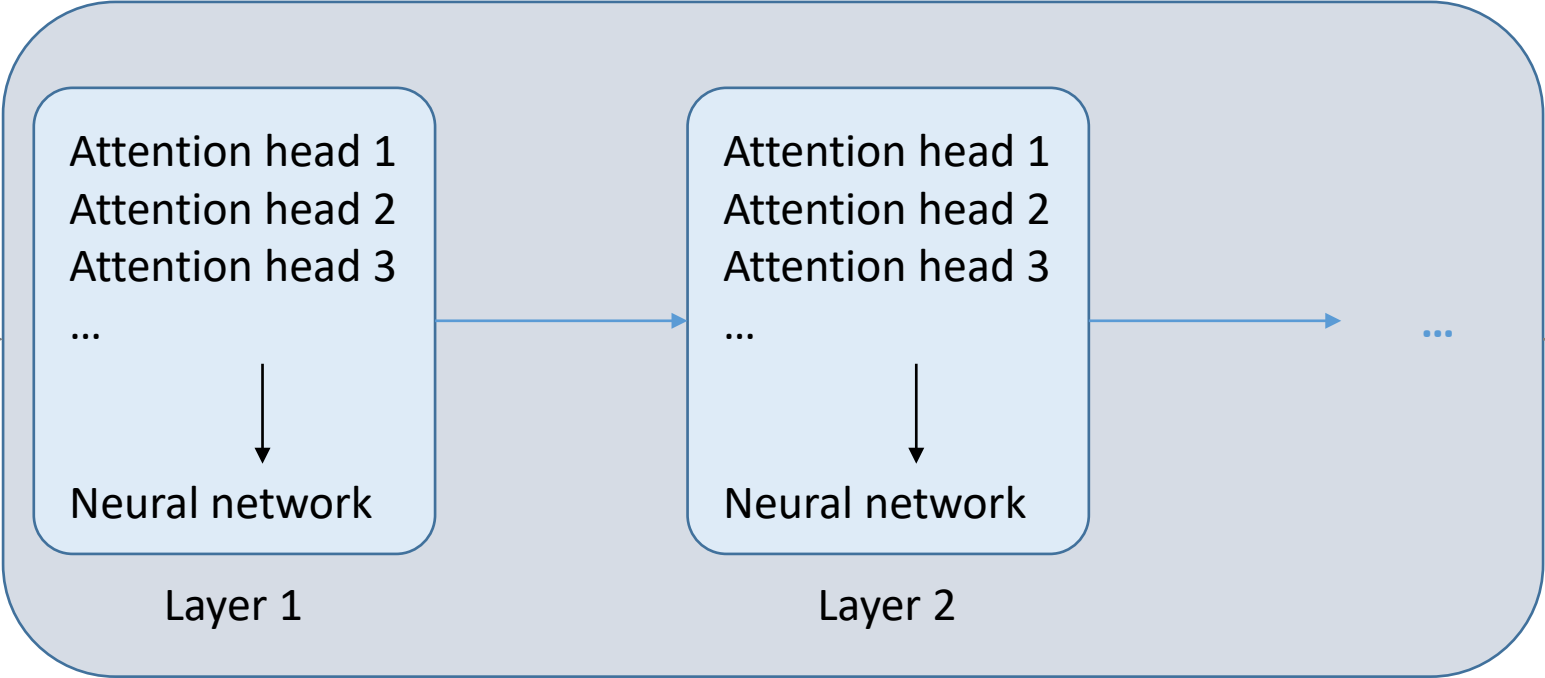
Example document: **Diagnostic fragment ions at m/z 204 indicate the presence of glycans.**

tokens → token embeddings → **transformer** → pooled vector

["The", "mitochondria", "is", "powerhouse"]

↓
Table lookup

↓
[v1, v2, v3, v4]



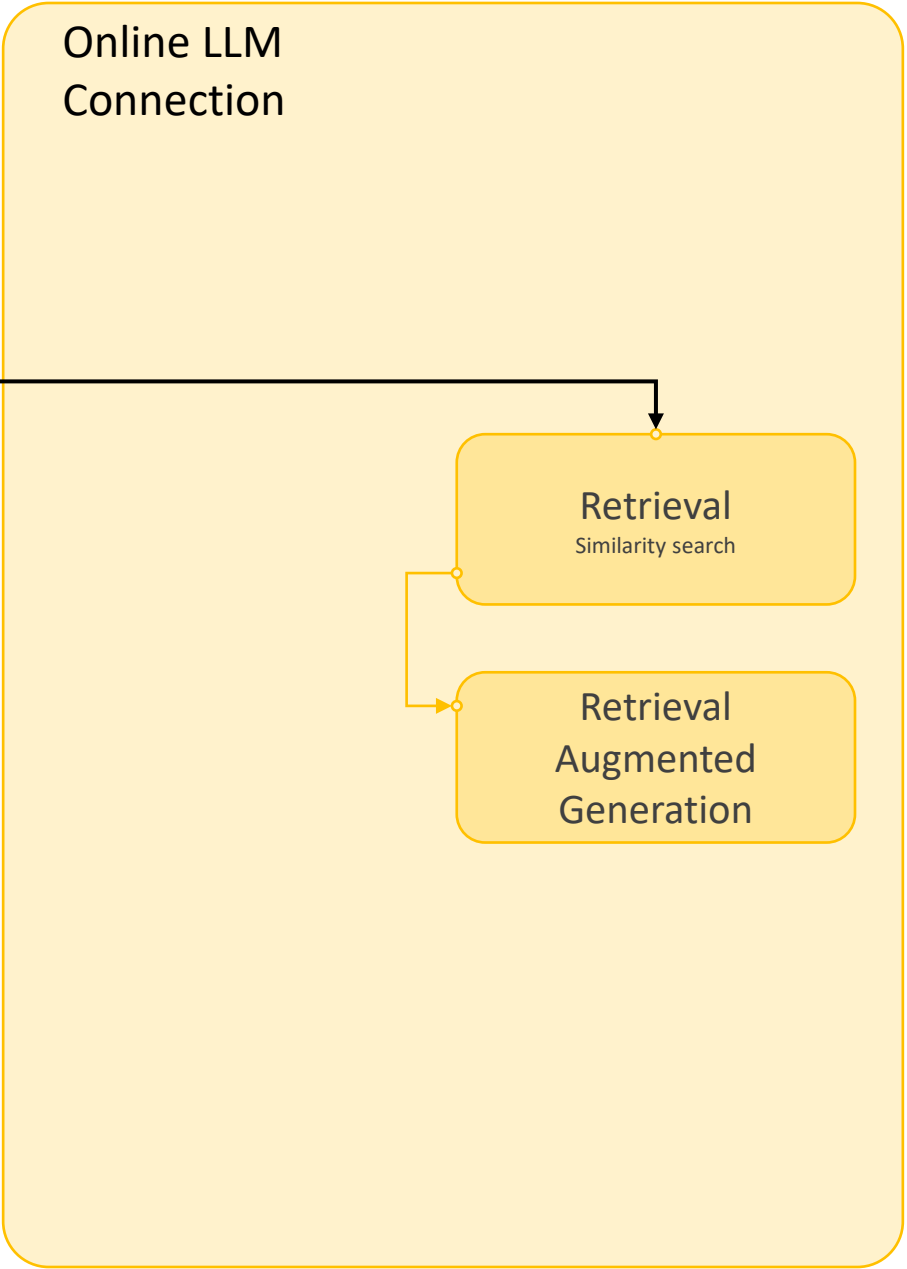
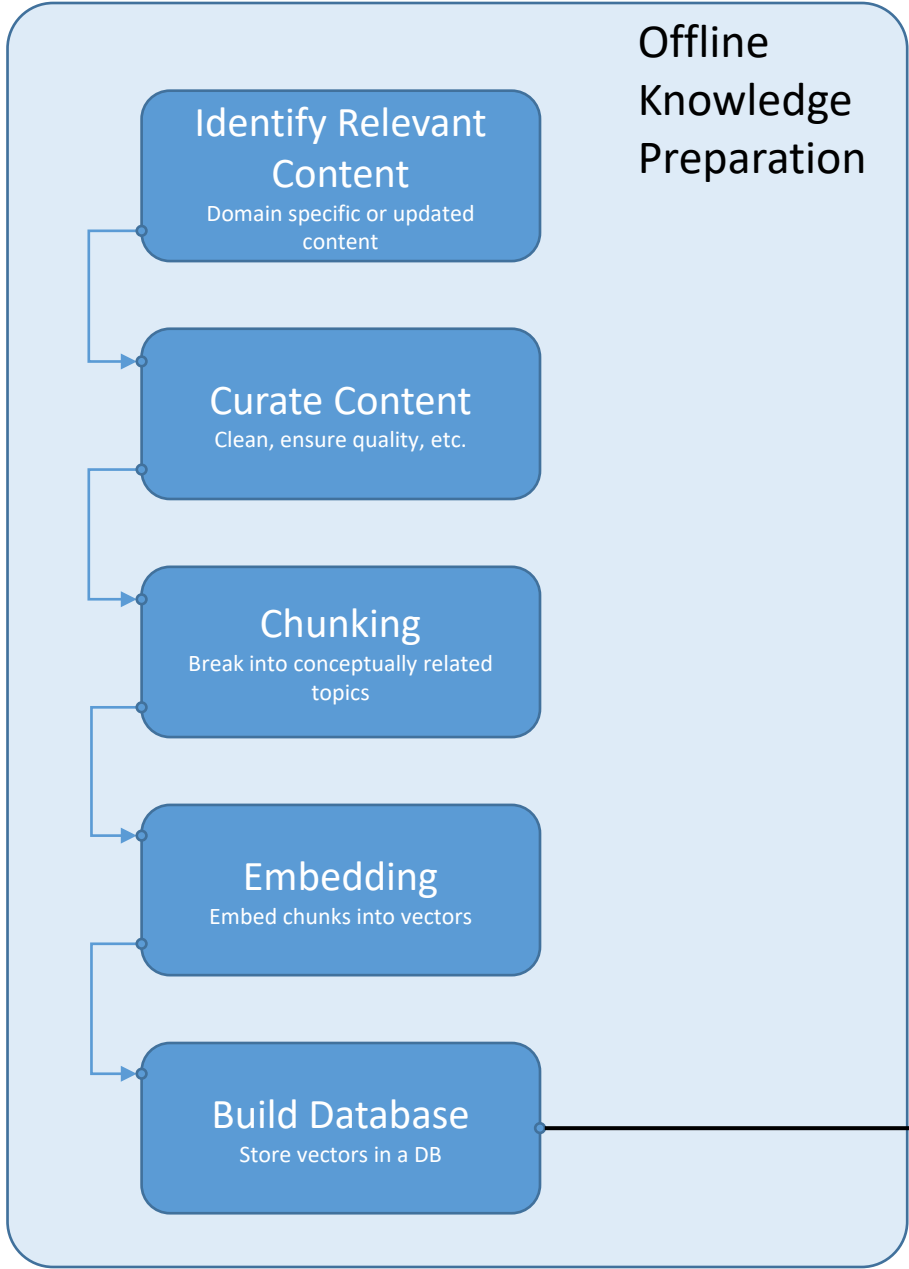
Transformer

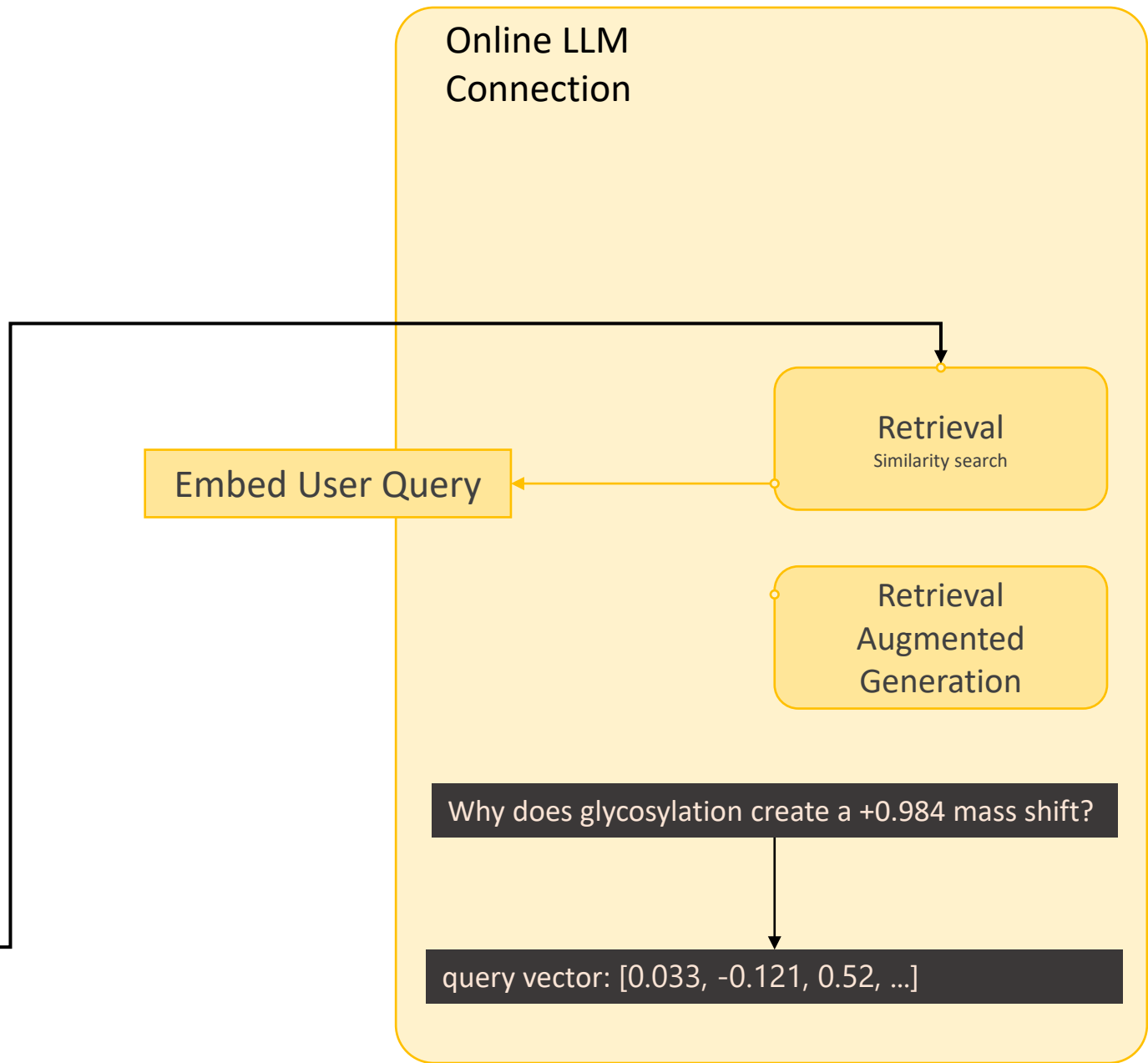
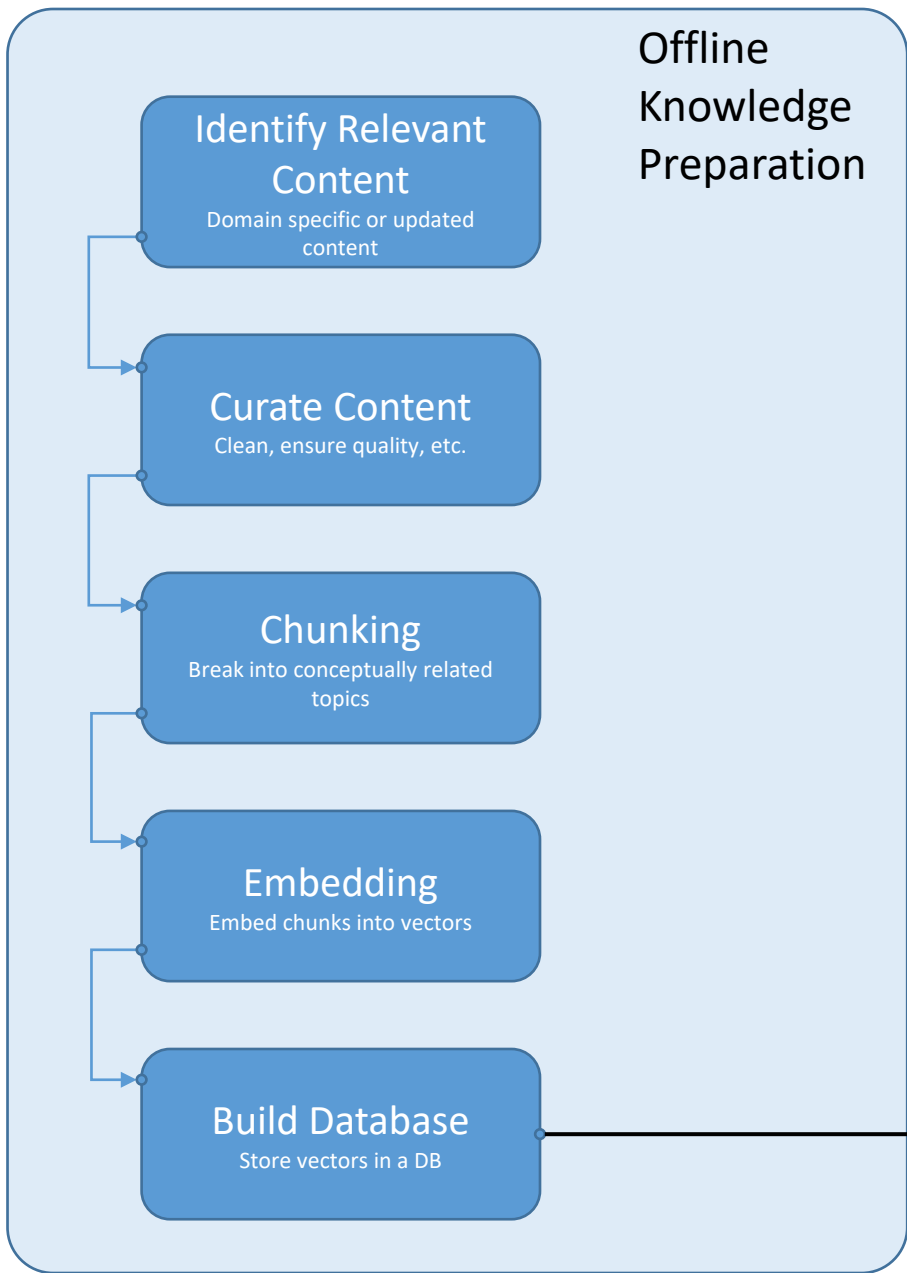
Contextualized vectors → Mean pooling → Pooled vectors

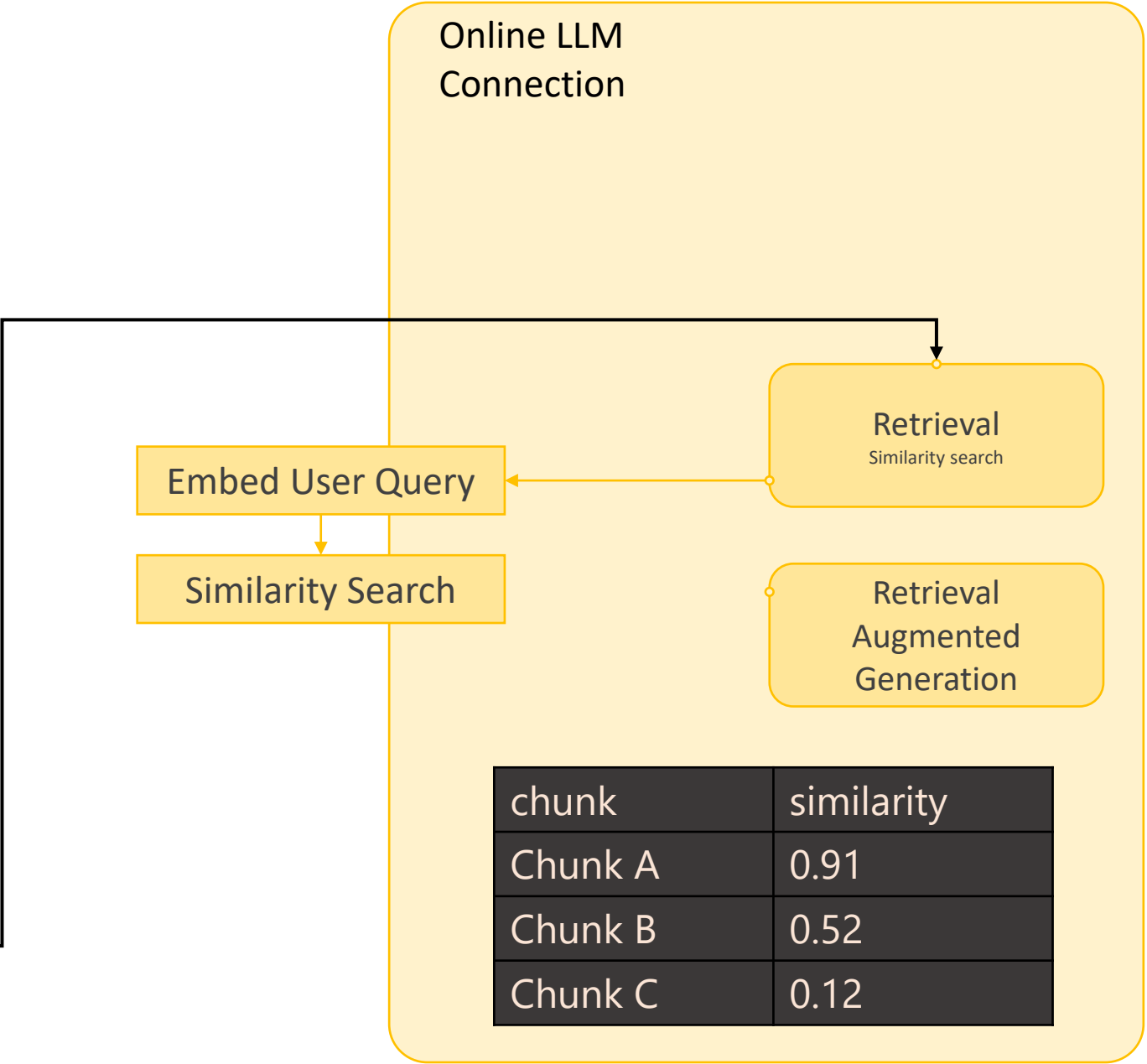
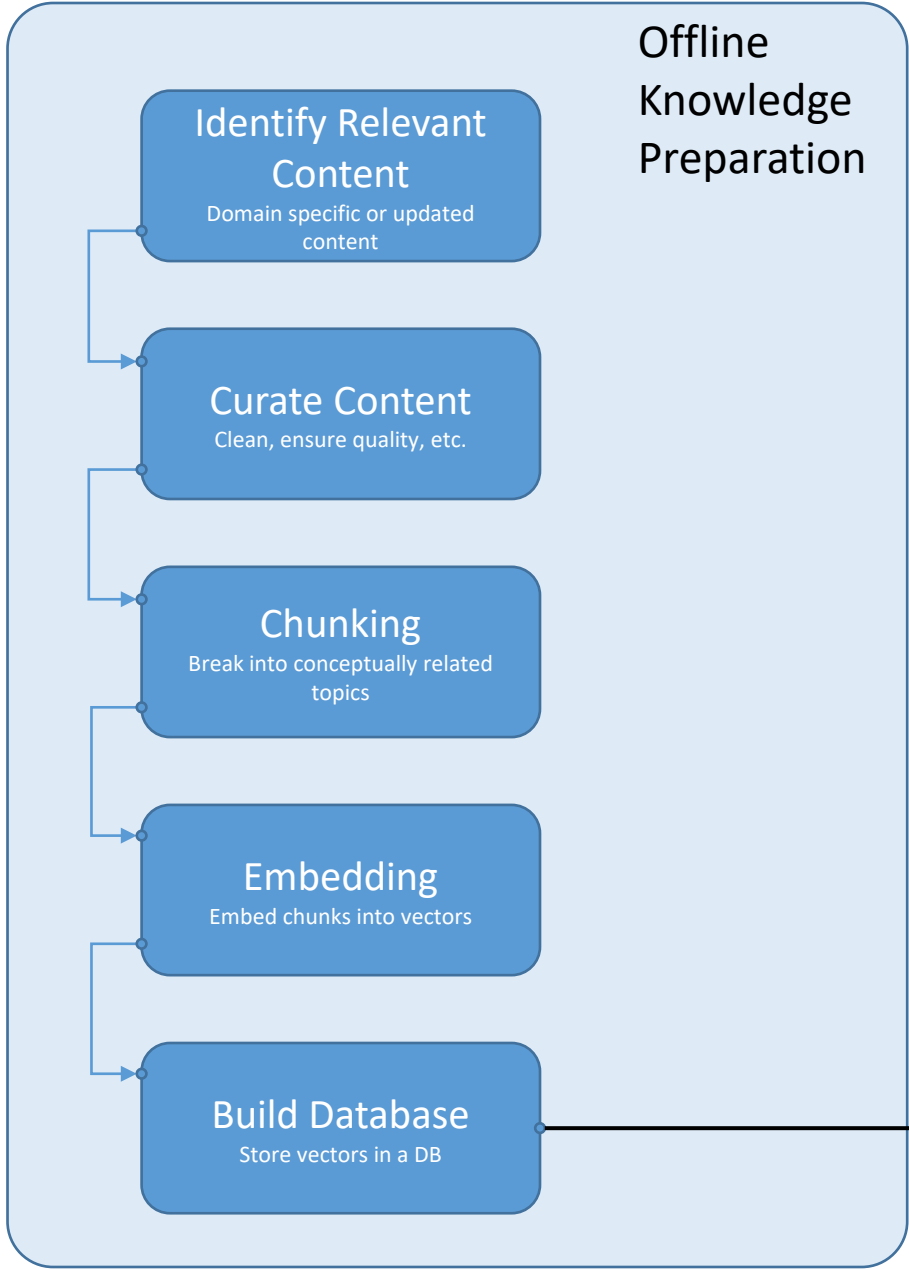
DB Contains Vector Values and Metadata

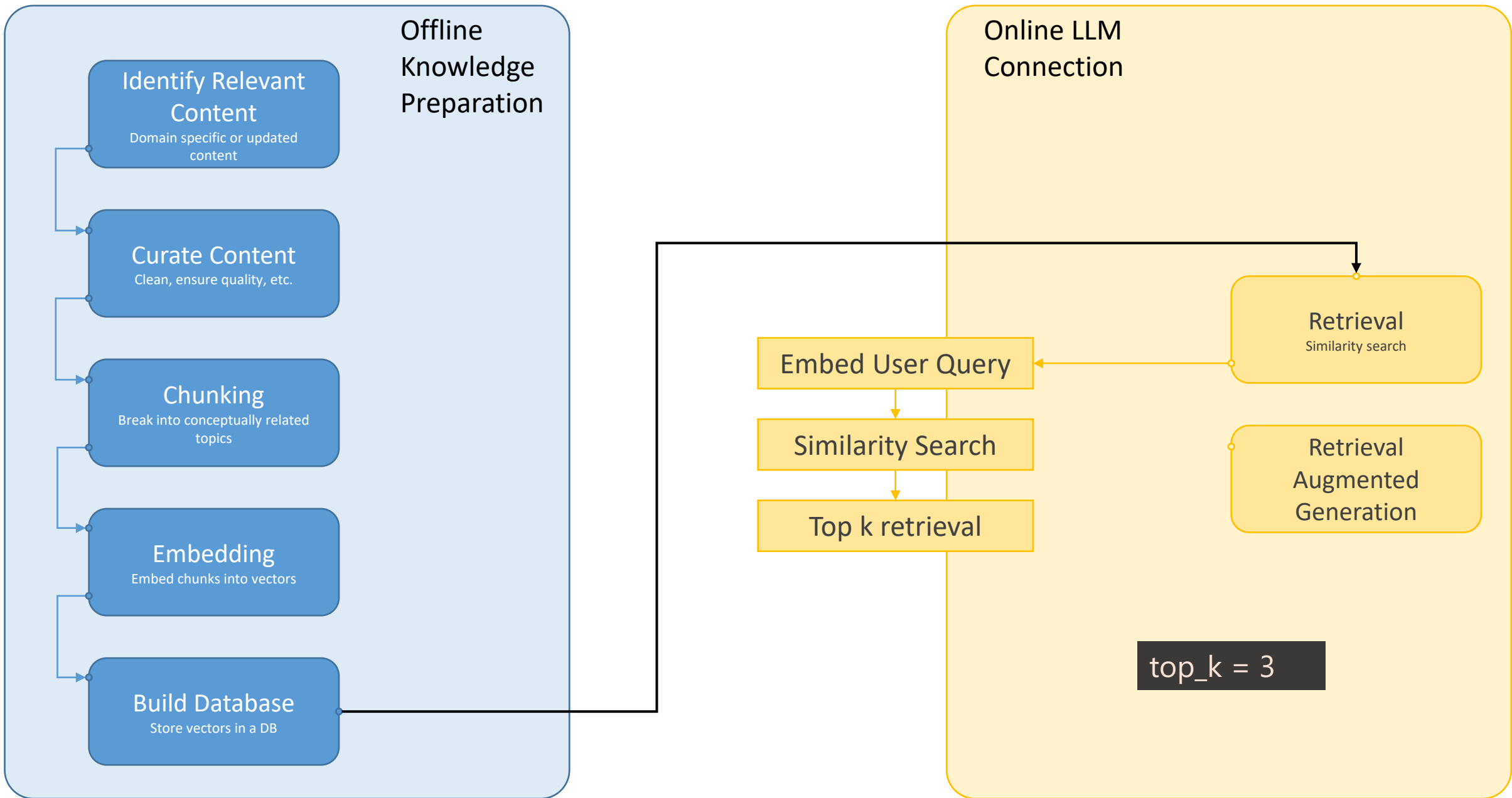
```
{  
  "vector": [0.021, -0.114, 0.55, ...],  
  "text": "chunk text",  
  "source": "paper_name",  
  "section": "methods"  
}
```

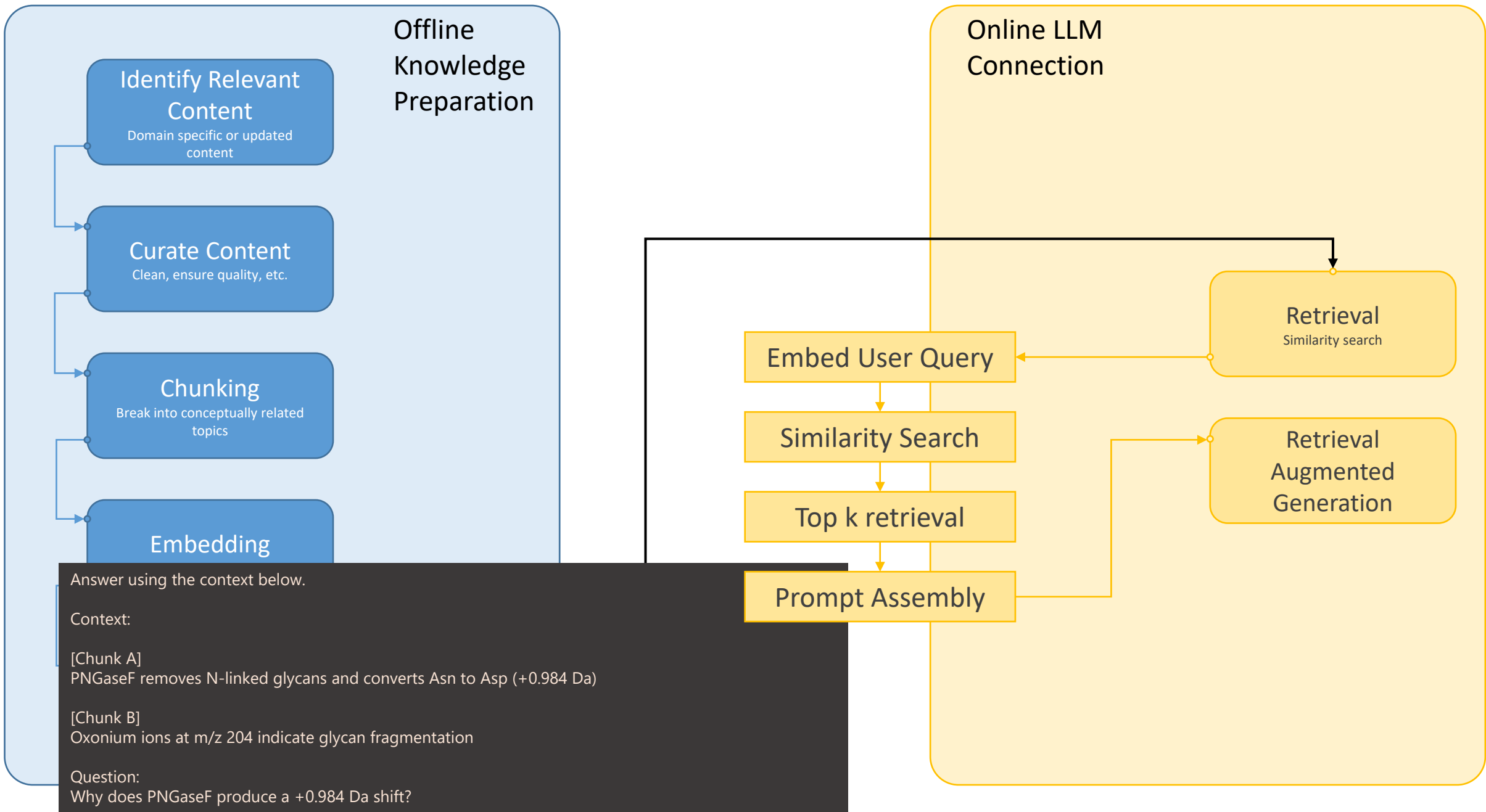
E.g. FAISS, Chroma, Pinecone, Weaviate, Milvus

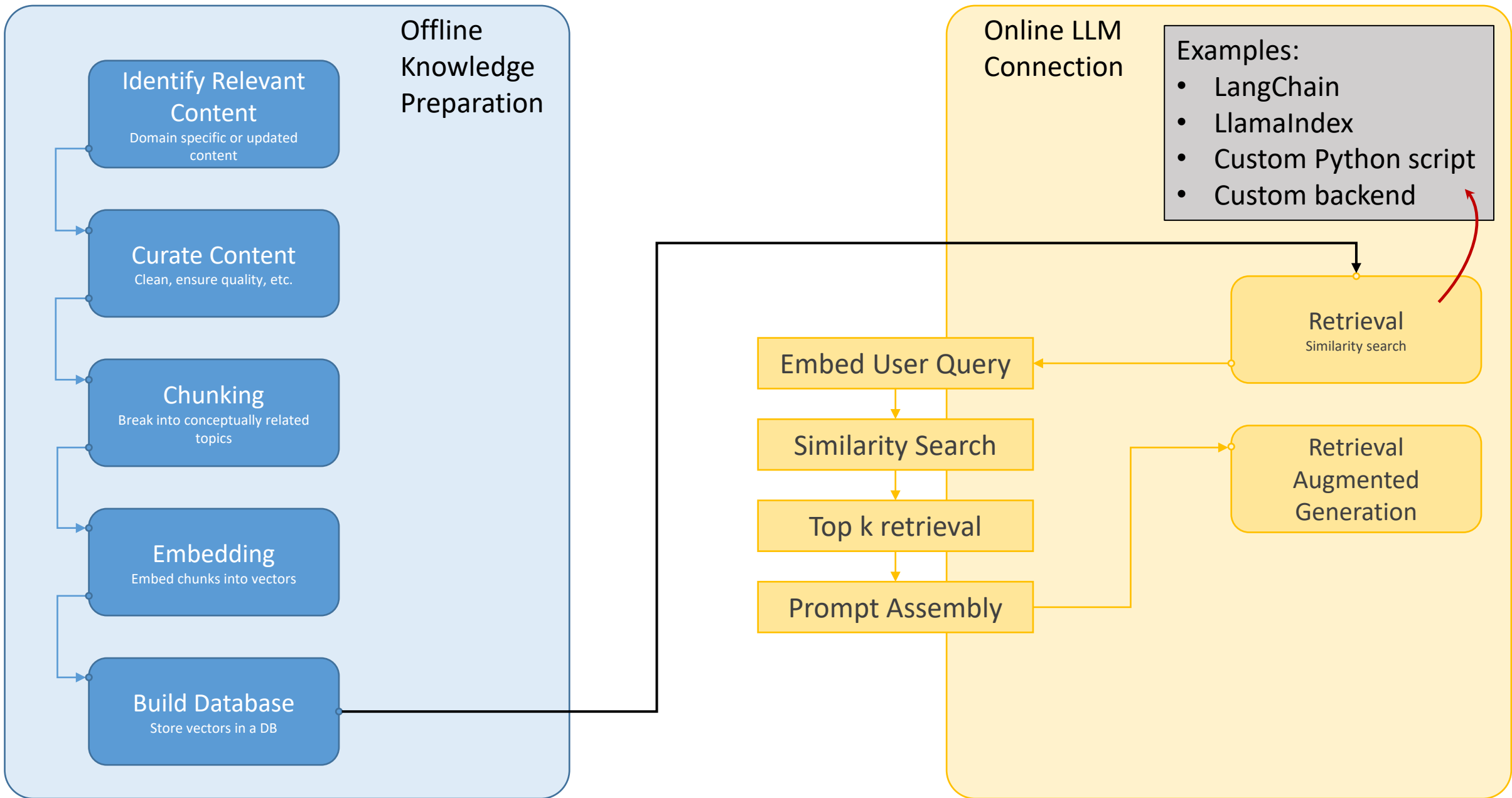


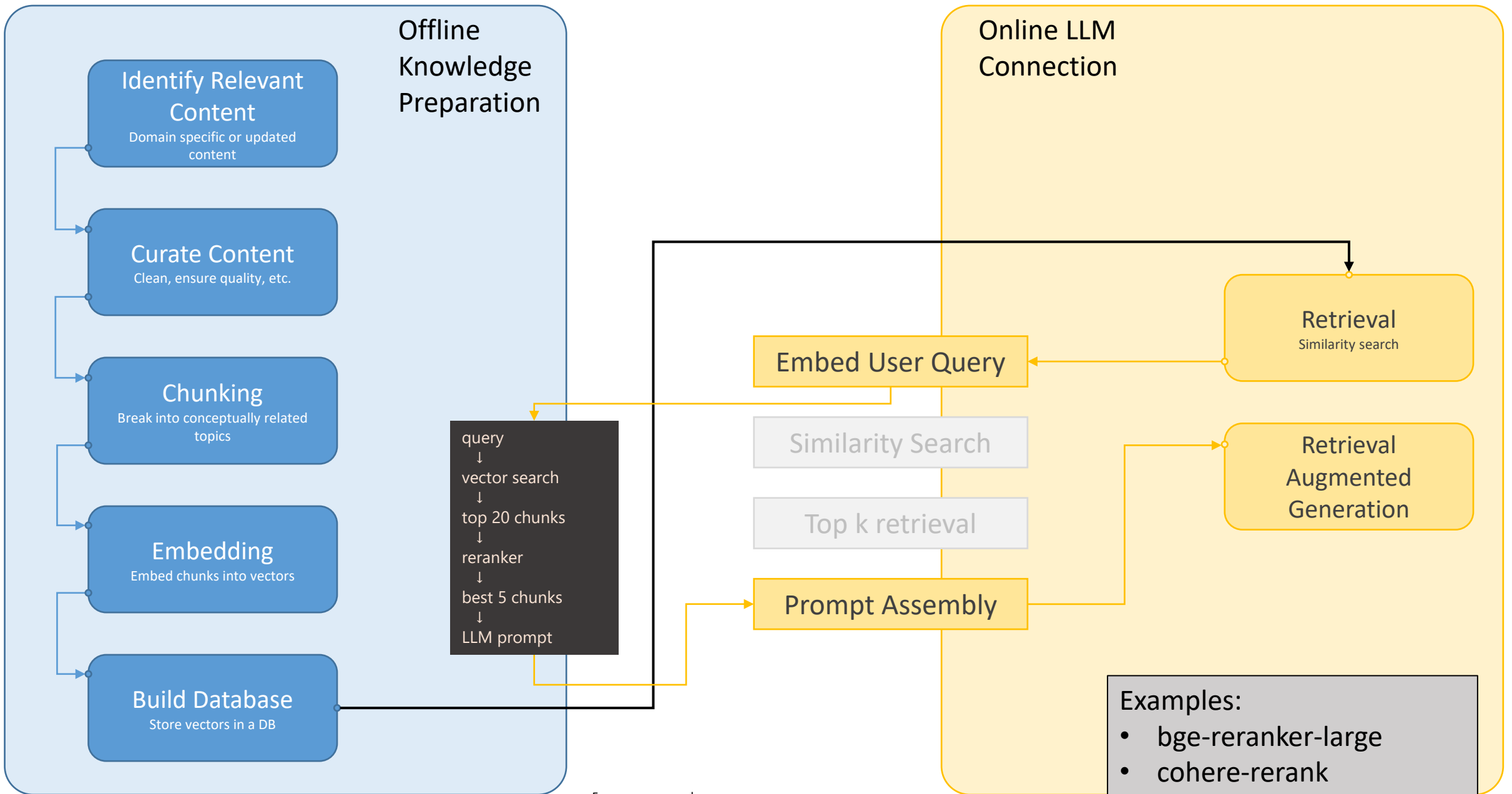








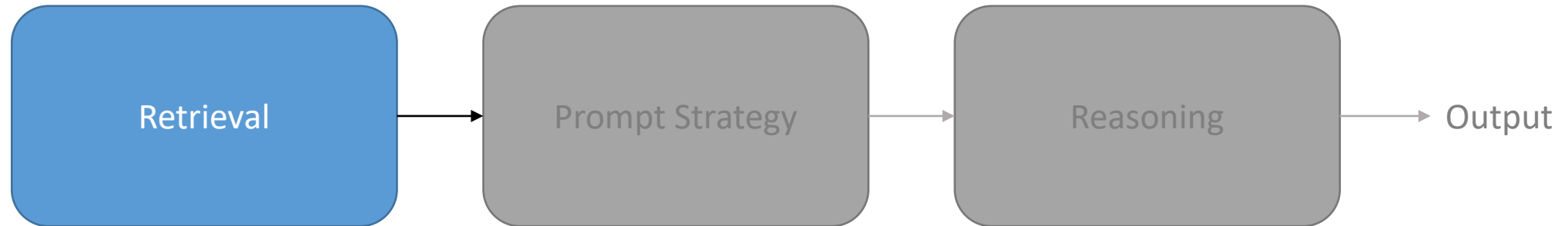




For more on rerankers, see:
<https://www.pinecone.io/learn/series/rag/rerankers/#Power-of-Rerankers>
<https://www.zeroentropy.dev/articles/what-is-a-reranker-and-do-i-need-one>

Testing

- Build tiny corpus of representative documents
- Write domain-specific queries
- For each query, define the chunk you believe should be retrieved
- Run retrieval with embedding model A, then B, then C...
- Compare where the correct chunk appears



Testing

- Test embedding models, e.g.:
 - text-embedding-3-small
 - text-embedding-3-large
 - bge-large
 - e5-large
- Test retrieval strategy, e.g.:
 - Vector only
 - Vector + keyword (hybrid)
 - Vector + reranker
- Test chunking strategy, e.g.:
 - fixed-size chunks
 - semantic chunks
 - section-based chunks

Testing

- Build a test matrix:

config	embedding	chunking	retrieval
A	small	fixed	vector
B	large	semantic	vector
C	large	semantic	+ reranker

script1.py

<https://github.com/GWU-Bioinformatics/2026Spring-BIOC-6225/tree/main/lecture-9>

```

import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]

def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)

def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
    #return corpus_of_documents[similarities.index(max(similarities))]

user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []

prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""

url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama2",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}

headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))

```

Start with Bill Chambers' code

```

import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]

def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)

def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
    #return corpus_of_documents[similarities.index(max(similarities))]

user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []

prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""

url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama2",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}

headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))

```

Our curated data!

```

import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]

def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)

def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
    #return corpus_of_documents[similarities.index(max(similarities))]

user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []

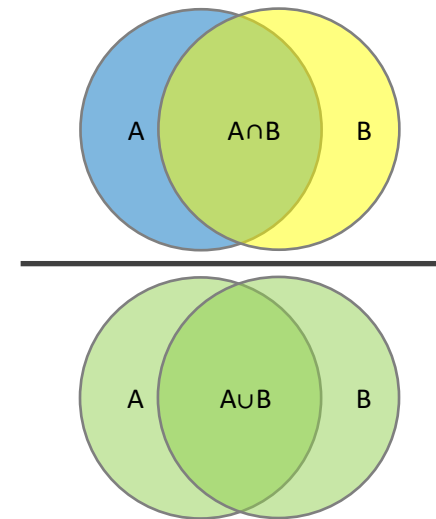
prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""

url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama2",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}

headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))

```



```

import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]

def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)

def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
    #return corpus_of_documents[similarities.index(max(similarities))]

user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []

prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""

url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama2",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}

headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))

```

- Take user query and all documents as input
- Perform similarity search using Jaccard
- Pick the best match

```
import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]
```

```
def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)
```

```
def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
#return corpus_of_documents[similarities.index(max(similarities))]
```

```
user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []
```

```
prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""
```

```
url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama3",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}
```

```
headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))
```

Prompt injection

```

import requests
import json

corpus_of_documents = [
    "Take a leisurely walk in the park and enjoy the fresh air.",
    "Visit a local museum and discover something new.",
    "Attend a live music concert and feel the rhythm.",
    "Go for a hike and admire the natural scenery.",
    "Have a picnic with friends and share some laughs.",
    "Explore a new cuisine by dining at an ethnic restaurant.",
    "Take a yoga class and stretch your body and mind.",
    "Join a local sports league and enjoy some friendly competition.",
    "Attend a workshop or lecture on a topic you're interested in.",
    "Visit an amusement park and ride the roller coasters."
]

def jaccard_similarity(query, document):
    query = query.lower().split(" ")
    document = document.lower().split(" ")
    intersection = set(query).intersection(set(document))
    union = set(query).union(set(document))
    return len(intersection)/len(union)

def return_response(query, corpus):
    similarities = []
    for doc in corpus:
        similarity = jaccard_similarity(query, doc)
        similarities.append(similarity)
    return corpus[similarities.index(max(similarities))]
    #return corpus_of_documents[similarities.index(max(similarities))]

user_input = "I like to hike"
relevant_document = return_response(user_input, corpus_of_documents)
full_response = []

prompt = """
You are a bot that makes recommendations for activities. You answer in very short sentences and do not include extra information.
This is the recommended activity: {relevant_document}
The user input is: {user_input}
Compile a recommendation to the user based on the recommended activity and the user input.
"""

url = 'http://localhost:11434/api/generate'
data = {
    "model": "llama3",
    "prompt": prompt.format(user_input=user_input, relevant_document=relevant_document)
}

headers = {'Content-Type': 'application/json'}
response = requests.post(url, data=json.dumps(data), headers=headers, stream=True)
try:
    count = 0
    for line in response.iter_lines():
        if line:
            decoded_line = json.loads(line.decode('utf-8'))

            full_response.append(decoded_line['response'])
finally:
    response.close()
print(''.join(full_response))

```

API infrastructure

Run the Code!

script1.py

script2.py

<https://github.com/GWU-Bioinformatics/2026Spring-BIOC-6225/tree/main/lecture-9>

Improve the Code

Update imports:

```
import json
import math
import re
from collections import Counter
import requests
```

Improve the Code

Add stopwords:

```
STOPWORDS = {  
    "a", "an", "and", "are", "as", "at", "be", "by", "for", "from",  
    "has", "have", "i", "in", "is", "it", "its", "of", "on", "or", "that",  
    "the", "to", "was", "were", "will", "with", "you", "your"  
}
```

Improve the Code

Add tokenization:

```
query = query.lower().split(" ")  
document = document.lower().split(" ")
```



```
def tokenize(text):  
    tokens = re.findall(r"\b\w+\b", text.lower())  
    return [token for token in tokens if token not in STOPWORDS]
```

Improve the Code

Remove Jaccard Similarity:

```
def jaccard_similarity(query, document):  
    query = query.lower().split(" ")  
    document = document.lower().split(" ")  
    intersection = set(query).intersection(set(document))  
    union = set(query).union(set(document))  
    return len(intersection)/len(union)
```

Improve the Code

Add TF-IDF computation:

```
def compute_idf(corpus):
    num_docs = len(corpus)
    document_frequency = Counter()

    for doc in corpus:
        unique_tokens = set(tokenize(doc))
        for token in unique_tokens:
            document_frequency[token] += 1

    idf = {}
    for token, df in document_frequency.items():
        idf[token] = math.log((num_docs + 1) / (df + 1)) + 1

    return idf
```

Improve the Code

Add TF-IDF vector builder:

```
def compute_tfidf_vector(text, idf):
    tokens = tokenize(text)
    if not tokens:
        return {}

    term_counts = Counter(tokens)
    total_terms = len(tokens)

    vector = {}
    for term, count in term_counts.items():
        tf = count / total_terms
        term_idf = idf.get(term, math.log((len(corpus_of_documents) + 1) / 1) + 1)
        vector[term] = tf * term_idf

    return vector
```

Improve the Code

Add cosine similarity:

Dot product {
$$\frac{||v1|| \cdot ||v2|| \cdot \cos\theta}{||v1|| \cdot ||v2||}$$

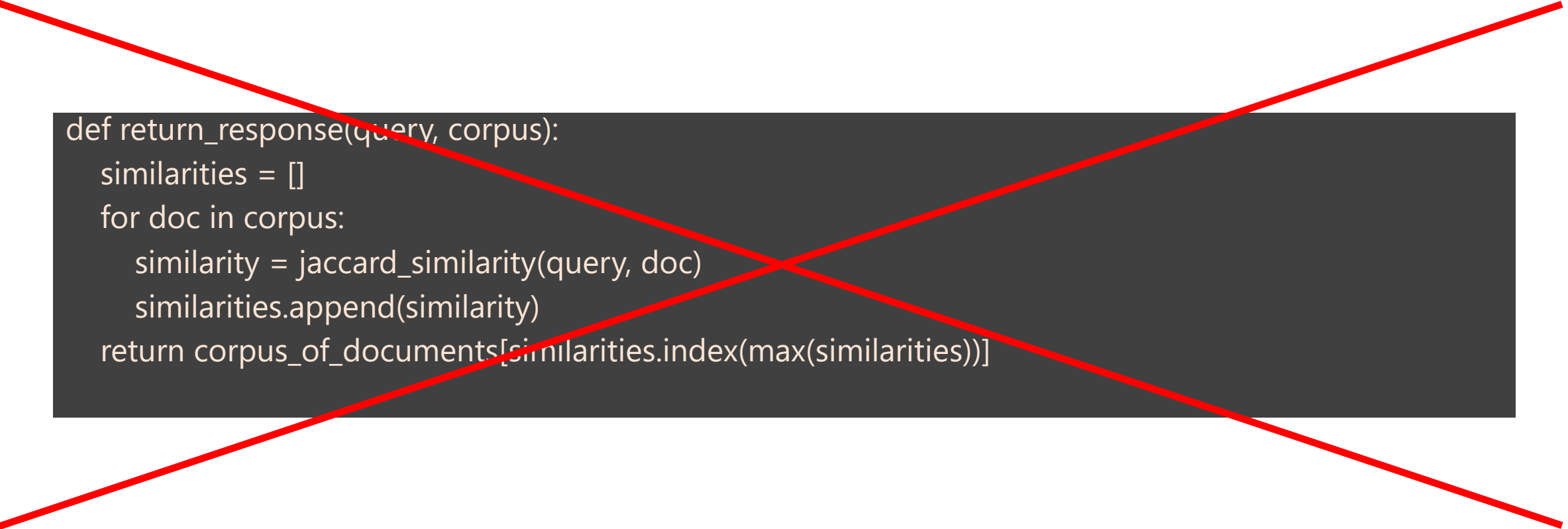
Magnitude of vector 1



```
def cosine_similarity(vec1, vec2):  
    if not vec1 or not vec2:  
        return 0.0  
  
    common_terms = set(vec1.keys()) & set(vec2.keys())  
    dot_product = sum(vec1[term] * vec2[term] for term in common_terms)  
  
    norm1 = math.sqrt(sum(weight * weight for weight in vec1.values()))  
    norm2 = math.sqrt(sum(weight * weight for weight in vec2.values()))  
  
    if norm1 == 0.0 or norm2 == 0.0:  
        return 0.0  
  
    return dot_product / (norm1 * norm2)
```

Improve the Code

Remove top scoring retrieval step:



```
def return_response(query, corpus):  
    similarities = []  
    for doc in corpus:  
        similarity = jaccard_similarity(query, doc)  
        similarities.append(similarity)  
    return corpus_of_documents[similarities.index(max(similarities))]
```

Improve the Code

Add top_k hits:

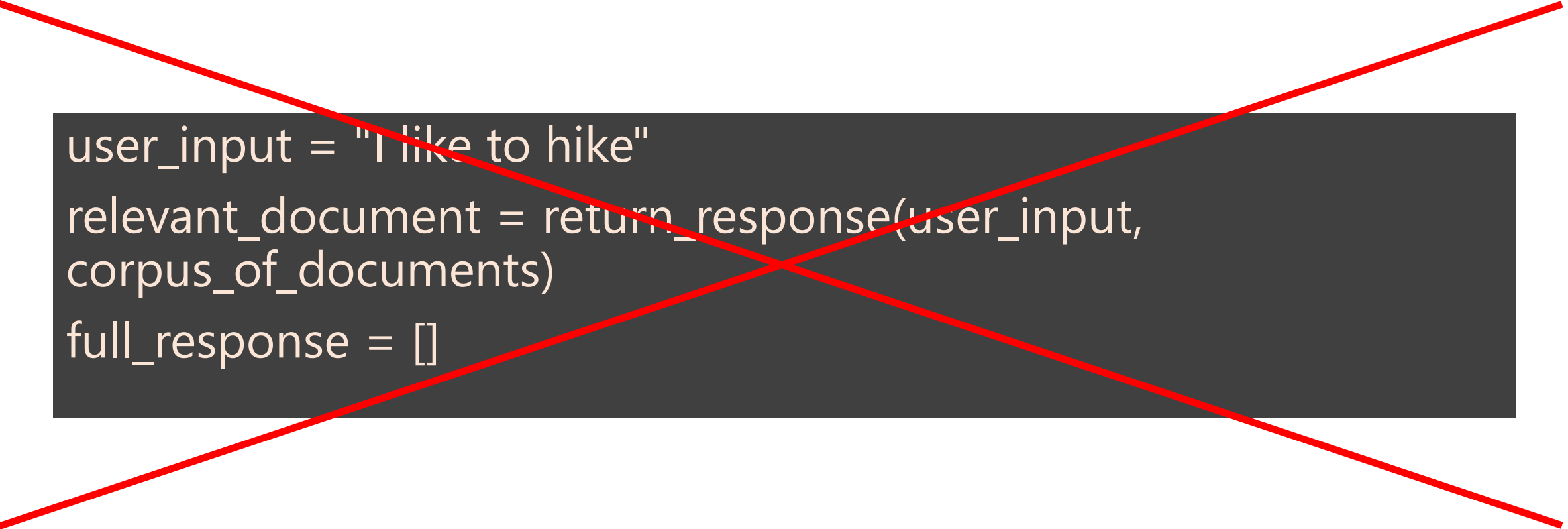
```
def retrieve_top_k(query, corpus, k=3, score_threshold=0.10):
    idf = compute_idf(corpus)
    query_vector = compute_tfidf_vector(query, idf)

    scored_documents = []
    for doc in corpus:
        doc_vector = compute_tfidf_vector(doc, idf)
        score = cosine_similarity(query_vector, doc_vector)
        if score >= score_threshold:
            scored_documents.append((doc, score))

    scored_documents.sort(key=lambda item: item[1], reverse=True)
    return scored_documents[:k]
```

Improve the Code

Remove single document line:



```
user_input = "I like to hike"  
relevant_document = return_response(user_input,  
corpus_of_documents)  
full_response = []
```

Improve the Code

Remove prompt handling:

```
prompt = """  
You are a bot that makes recommendations for activities. You answer in very  
short sentences and do not include extra information  
This is the recommended activity: {relevant_document}  
The user input is: {user_input}  
Compile a recommendation to the user based on the recommended activity  
and the user input.  
"""
```

Improve the Code

Update prompt builder:

```
def build_prompt(user_input, retrieved_docs):
    if not retrieved_docs:
        retrieved_text = "No relevant activities were retrieved."
    else:
        retrieved_lines = [
            f"- {doc} (score={score:.3f})"
            for doc, score in retrieved_docs
        ]
        retrieved_text = "\n".join(retrieved_lines)

    prompt = f"""
    You are a bot that makes recommendations for activities.
    You answer in very short sentences and do not include extra information.

    The user input is: {user_input}

    Retrieved activities:
    {retrieved_text}

    If relevant activities were retrieved, recommend the best one.
    If nothing relevant was retrieved, say that you need more detail from the user.
    """
    return prompt.strip()
```

Improve the Code

Update generation code:

```
def generate_response(prompt, model="llama2"):
    url = "http://localhost:11434/api/generate"
    data = {
        "model": model,
        "prompt": prompt
    }
    headers = {"Content-Type": "application/json"}

    full_response = []

    response = requests.post(
        url,
        data=json.dumps(data),
        headers=headers,
        stream=True
    )

    try:
        response.raise_for_status()

        for line in response.iter_lines():
            if line:
                decoded_line = json.loads(line.decode("utf-8"))
                full_response.append(decoded_line.get("response", ""))
    finally:
        response.close()

    return "".join(full_response)
```

Improve the Code

Add top level code:

```
if __name__ == "__main__":
    user_input = "I like to hike"

    top_docs = retrieve_top_k(
        query=user_input,
        corpus=corpus_of_documents,
        k=3,
        score_threshold=0.10
    )

    print("Top retrieved documents:")
    if top_docs:
        for rank, (doc, score) in enumerate(top_docs, start=1):
            print(f"{rank}. score={score:.3f} | {doc}")
    else:
        print("No documents passed the score threshold.")

    prompt = build_prompt(user_input, top_docs)

    print("\nGenerated response:")
    try:
        model_output = generate_response(prompt, model="llama2")
        print(model_output)
    except requests.RequestException as exc:
        print(f"Request failed: {exc}")
```

Summary

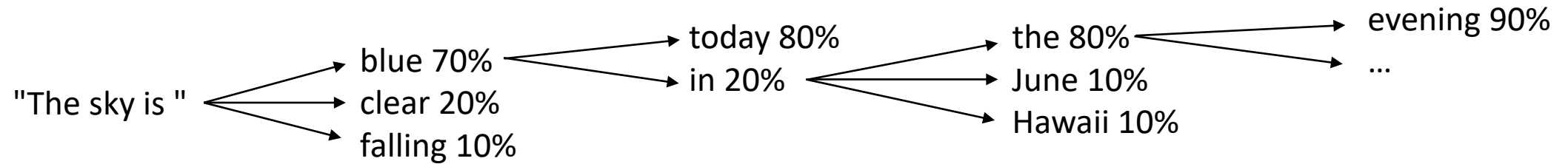
- Query
 - Tokenize/clean
 - Compute corpus IDF
 - Build TF-IDF query vector
 - Build TF-IDF doc vectors
 - Cosine similarity scoring
 - Threshold + top-k ranking
 - Prompt with retrieved docs
 - Model

Run the Code!

script2.py

Temperature and Seed

p (next token | previous tokens)



Force Reproducibility

```
data = {  
  "model": "llama3",  
  "prompt": prompt,  
  "options": {  
    "temperature": 1.2  
  }  
}
```

```
data = {  
  "model": "llama3",  
  "prompt": prompt,  
  "options": {  
    "temperature": 0.0,  
    "seed": 42  
  }  
}
```

script3.py

<https://github.com/GWU-Bioinformatics/2026Spring-BIOC-6225/tree/main/lecture-9>

scikit-learn

- This package now handles a large section of the code
 - Tokenization
 - Lowercasing
 - Stopword removal
 - TF-IDF weighting
 - Sparse matrix creation
 - Cosine similarity computation

Key Changes

```
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.metrics.pairwise import cosine_similarity
```

```
vectorizer = TfidfVectorizer(  
    lowercase=True,  
    stop_words="english",  
    token_pattern=r"(?u)\b\w+\b"  
)
```

```
doc_matrix = vectorizer.fit_transform(corpus)
```

```
query_vector = vectorizer.transform([query])
```

```
similarity_scores = cosine_similarity(query_vector, doc_matrix)[0]
```

script4.py

<https://github.com/GWU-Bioinformatics/2026Spring-BIOC-6225/tree/main/lecture-9>

Progression

- Lexical retrieval only
 - script 1
- Lexical retrieval plus generation
 - script 2
- Lexical retrieval plus generation (scikit-learn)
 - script 3
- Now: semantic
 - Embed the query
 - Embed each document
 - Compare in embedding space
 - Retrieve top-k
 - Pass retrieved context into the generator
 - Generate answer

Embedding

```
EMBED_MODEL = "embeddinggemma"  
GEN_MODEL = "llama3"
```

```
url = f"{OLLAMA_BASE_URL}/embed"  
payload = {  
    "model": model,  
    "input": text  
}
```

Dense-vector cosine similarity

```
dot_product = sum(a * b for a, b in zip(vec1, vec2))
```

Vector Density

Sparse vectors

- Very high dimensionality
 - ~10K-1M+
- Most values are zeroes
 - Each dimension corresponds to a specific word or token

[0, 0, 3.2, 0, 0, 0]

Dense vectors

- Smaller dimensionality
 - Commonly between 384 and 3072
- Usually no zeroes

[0.12, -0.83, 0.44, 0.91, -0.07, 0.33]

Retrieval

```
query_embedding = get_embedding(query)
doc_embedding = get_embedding(doc)
score = cosine_similarity(query_embedding, doc_embedding)
```

BCO-RAG Example



BCO-RAG Documentation

[Home](#)[Setup and Quickstart](#)[Output Structure](#)[Usage](#)[Other Features](#)[Code Documentation](#)

Biocompute Object Retrieval-Augmented Generation Assistant

Background

The BioCompute Object (BCO) project is a community-driven open standards framework for standardizing and sharing computations and analyses. With the exponential increase in both the quantity and complexity of biological data and the workflows used to analyze and transform the data, the need for standardization in documentation is imperative for experimental preservation, transparency, accuracy, and reproducibility.

As with any documentation standard, the main hurdles to continued adoption are the overhead required to maintain the quality and accuracy of a BCO in parallel as the research evolves over time and retroactively documenting pre-existing research. With the recent improvements in large language models (LLMs), the feasibility and utility of an automated BCO creation assistant is an intriguing use case.

Goal

The goal of this project is to reduce the overhead required in retroactively documenting pre-existing research. By using the Biocompute RAG assistant, you can seamlessly convert existing publications on previous research to be BCO compliant.

Approach Justification

This BioCompute Object (BCO) assistant will assist users in automatically creating specific BCO domains from user uploaded papers. This proof of concept uses a **Retrieval Augmented Generation (RAG)** approach rather than a standalone (or fine-tuned) LLM. Our use case is somewhat antithetical to what LLMs were originally designed for. LLMs were designed for creative, free text responses that represent plausible natural language. BCOs, on the other hand, were designed for deterministic, accurate, non-ambiguous, and reproduceable documentation. Given this, there are two main initial questions that have to be considered:

- Current LLMs are often categorized as "stochastic parrots" that have no underlying understanding of text structure, only generating what are considered plausible natural

Table of contents

[Background](#)[Goal](#)[Approach Justification](#)[Issues with Long Context](#)[Windows](#)[Training Data and False](#)[Extrapolation](#)

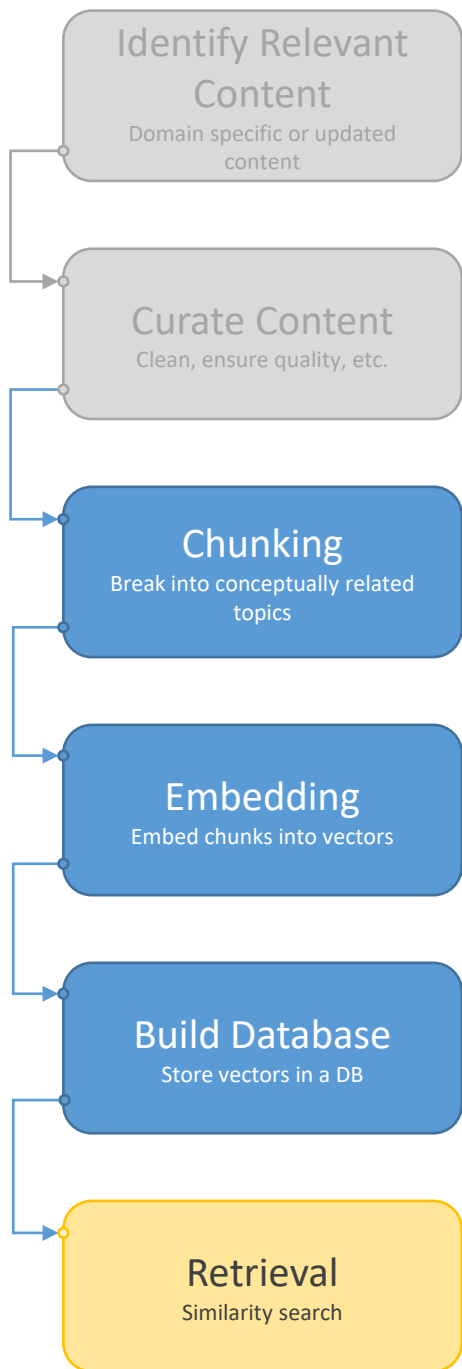
BCO-RAG

Repository:

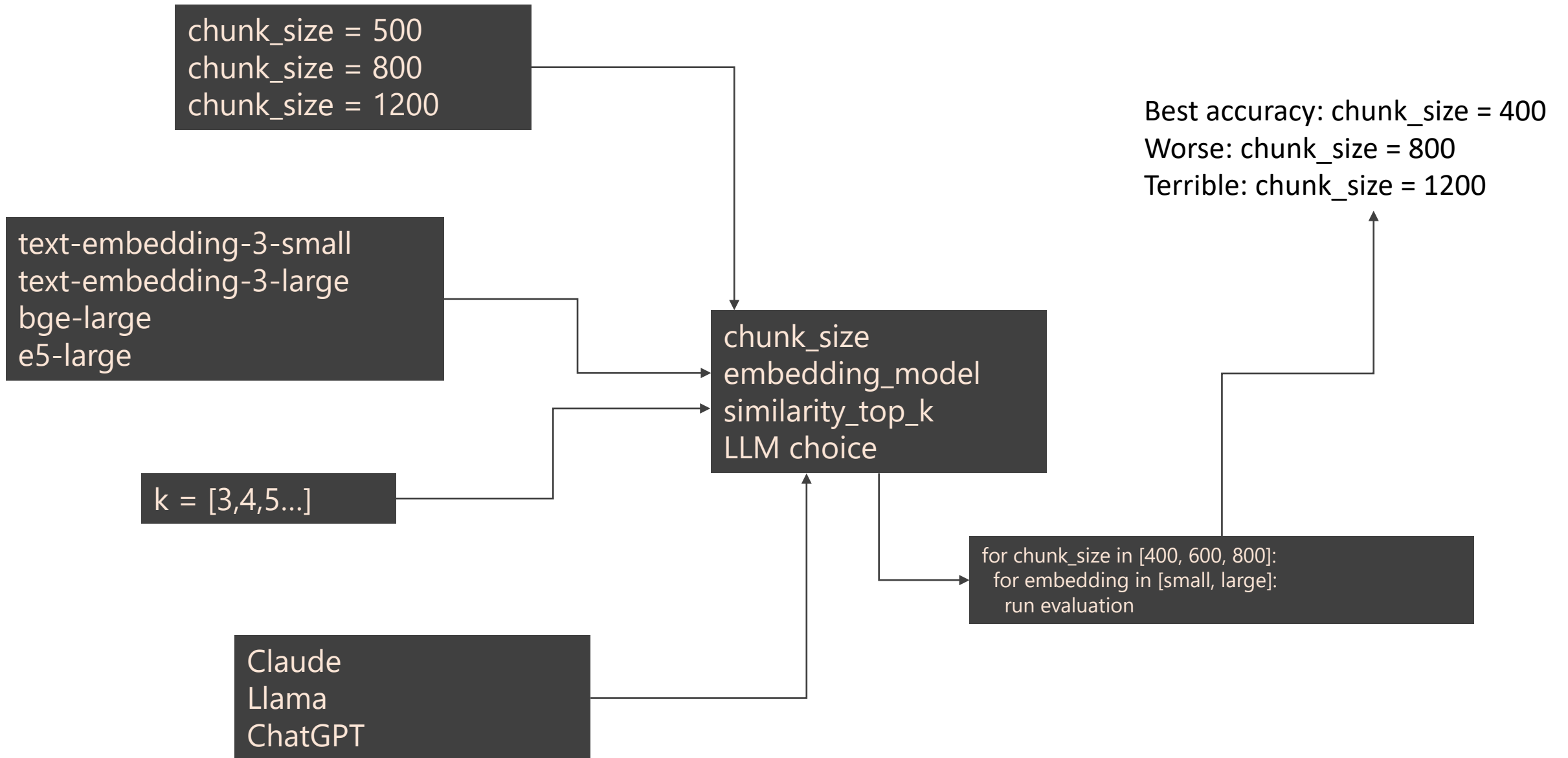
<https://github.com/biocompute-objects/bco-rag>

Documentation:

<https://biocompute-objects.github.io/bco-rag/>**Chunking:** size-based**Embeddings:** sentence transformer**DB:** In memory (no production DB)**Retrieval:** cosine similarity/top_k**Prompt packaging:** simple context stuffing



script4.py	bco-rag
No chunking	Fixed-size chunking
API call to Ollama	Local Python model (numpy)
Build dict object one by one	Build tuples in batch
Cosine similarity/threshold/top_k	Cosine similarity/top_k



Systematically test many options

Summary

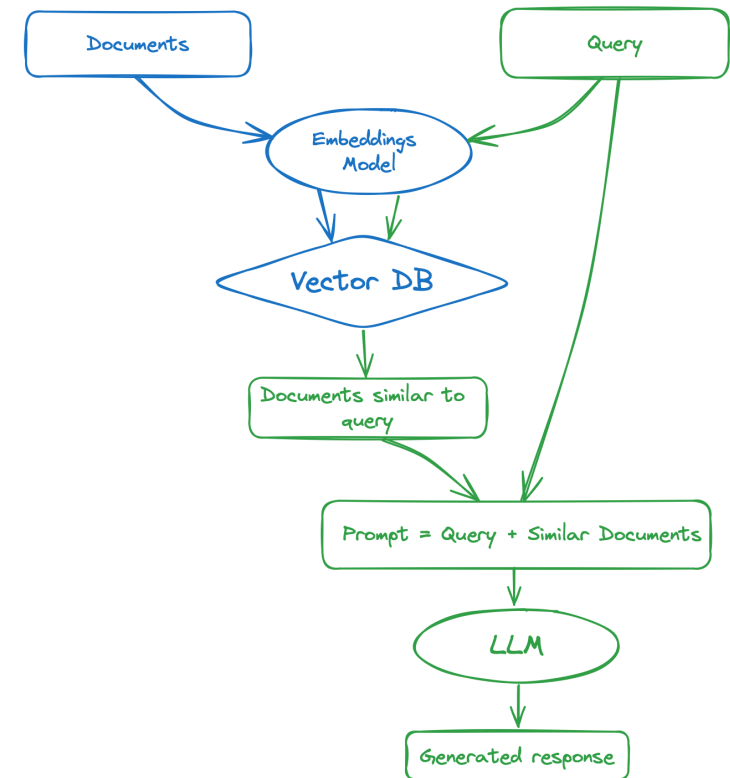
Key Concepts for Retrieval Augmentation

- 1. RAGs are powerful ways of extending LLMs with structured knowledge**
- 2. An LLM does not see your entire database**
- 3. A good LLM cannot fix upstream errors**
- 4. Good retrieval + small LLM > bad retrieval + large LLM**
- 5. Components are fungible**

Next Steps

- See Huggingface implementation

AutoTokenizer
HuggingFaceEmbeddings
FAISS DB
Zephyr-7b-beta
langchain



https://huggingface.co/learn/cookbook/rag_zephyr_langchain